

DSA Day 25: AVL Trees - rotations & insertion

Book: Data Structures Through C in Depth - Ch6 §6.14.1-6.14.3 · pp 225-247 | Code in Python

Overview

AVL trees keep $|\text{balance factor}| \leq 1$ using LL, RR, LR, RL rotations after insertion, guaranteeing $O(\log n)$.

Key concepts to master

- Balance factor = $h(\text{left}) - h(\text{right})$
- 4 rotation cases
- Rebalance up the path

Python implementation

```
def rotate_right(y):  
    x=y.left; y.left=x.right; x.right=y  
    return x # update heights after
```

Complexity

Search/insert/delete $O(\log n)$.

Common pitfalls

- Not updating heights after rotation
- Picking the wrong rotation case

Practice

- Insert 10,20,30 (RR) then 30,20,10 (LL)
- Identify the rotation for each imbalance

